

Exercise 4: The Rage-Click Defender (exhaustMap)

Context: Managing asynchronous event traffic requires careful control over concurrent HTTP requests. When a user interacts with a network action iteratively, selecting the wrong asynchronous flattening operator can cause server data corruption or duplicated transaction side effects.

The Three Flattening Operators — When Each One Matters

Before writing code, understand how the three primary RxJS flattening operators handle the scenario where a new user event arrives while a previous HTTP request is still actively in flight:

Operator	What It Does with the Old Request	Best Production Use Case
switchMap	Cancels the old request instantly, starts the new one.	Search Typeaheads: Cancels the slow, outdated Smith search string query the moment the user types Smith .
exhaustMap	Ignores the new emission completely until the old one finishes.	Submit Action Buttons: Safely drops rapid rage-clicks while the primary creation POST request is still pending.
concatMap	Queues the new request to execute after the old one finishes.	Sequential Data Syncs: Processes ledger adjustments or stream updates in strict, sequential queue order.

For Dawit’s grade submission form, the correct architectural choice is **exhaustMap**. While the first **POST** network transaction is in flight, any subsequent form submissions are dropped. This guarantees the grade is saved exactly once.

Security Warning: Using **switchMap** here is a dangerous anti-pattern. It cancels the in-flight request on the client browser frame, but the backend server may have already processed and committed the data write before the cancellation signal arrives across the network socket—resulting in a saved record with zero client confirmation.

[!NOTE]

Step 1: Generate the Component and Service Layer

Open a terminal inside your Angular workspace and execute the schema generators:

```
ng generate service services/grade --type=service
ng generate component features/grade-submission --type=component
```

[!NOTE]

Step 2: Implement the Grade Service

Open `src/app/services/grade.service.ts` and implement the HTTP client interface using Angular 22's `@Service()` decorator:

```
import { Service, inject } from "@angular/core";
import { HttpClient } from "@angular/common/http";
import { Observable } from "rxjs";

public interface GradePayload {
  studentId: number;
  courseId: number;
  score: number;
}

@Service()
export class GradeService {
  private http = inject(HttpClient);

  postGrade(payload: GradePayload): Observable<{ id: string;
  success: boolean }> {
    return this.http.post<{ id: string; success: boolean }>
    ('/api/grades', payload);
  }
}
```

[!NOTE]

Step 3: Implement the Guarded Component Class (Reactive Form)

Open `src/app/features/grade-submission/grade-submission.component.ts`. Import `ReactiveFormsModule`, `FormBuilder`, and `Validators` alongside Angular Material components. Construct an explicit `gradeForm` group and set up the `Subject`-based event stream protected by `exhaustMap`:

```
import { Component, inject } from '@angular/core';
import { FormBuilder, Validators, ReactiveFormsModule } from
 '@angular/forms';
import { Subject } from 'rxjs';
import { exhaustMap } from 'rxjs/operators';
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';
import { MatCardModule } from '@angular/material/card';
import { MatFormFieldModule } from '@angular/material/form-field';
import { MatInputModule } from '@angular/material/input';
import { MatProgressSpinnerModule } from
 '@angular/material/progress-spinner';
import { MatButtonModule } from '@angular/material/button';
import { GradeService, GradePayload } from
 '../services/grade.service';

@Component({
  selector: 'tms-grade-submission',
```

```

standalone: true,
imports: [
  ReactiveFormsModule,
  MatCardModule,
  MatFormFieldModule,
  MatInputModule,
  MatProgressSpinnerModule,
  MatButtonModule
],
templateUrl: './grade-submission.component.html'
})
export class GradeSubmissionComponent {
  private api = inject(GradeService);
  private fb = inject(FormBuilder);

  // Reactive Form definition with initial model values and
  validators
  gradeForm = this.fb.group({
    studentId: [101, [Validators.required, Validators.min(1)]],
    courseId: [302, [Validators.required, Validators.min(1)]],
    score: [88, [Validators.required, Validators.min(0),
Validators.max(100)]]
  });

  isSubmitting = false;
  submissionStatus = '';

  // A Subject is a manual event stream – template clicks push
  payloads into it
  private submitClick$ = new Subject<GradePayload>();

  constructor() {
    this.submitClick$
      .pipe(
        // exhaustMap: while the inner HTTP observable is active,
        // ALL new emissions from submitClick$ are silently
        dropped.
        // Dawit can click 50 times – only ONE POST request fires.
        exhaustMap(payload => {
          this.isSubmitting = true;
          this.submissionStatus = 'Submitting grade to server...';
          return this.api.postGrade(payload);
        }),
        // takeUntilDestroyed: automatically unsubscribes when
        Angular
        // destroys this component, preventing memory leaks.
        // Placed inside constructor to inherit the active
        injection context.
        takeUntilDestroyed()
      )
      .subscribe({
        next: result => {
          this.isSubmitting = false;

```

```

        this.submissionStatus = `Grade saved successfully! Record
ID: ${result.id}`;
    },
    error: err => {
        this.isSubmitting = false;
        this.submissionStatus = `Submission failed: ${err.message}
|| 'Server error'`;
    }
});
}

// The template form submit handler pushes valid values into the
protected stream
onSubmit() {
    if (this.gradeForm.valid) {
        const rawValue = this.gradeForm.getRawValue();
        this.submitClick$.next({
            studentId: Number(rawValue.studentId),
            courseId: Number(rawValue.courseId),
            score: Number(rawValue.score)
        });
    }
}
}
}

```

[!NOTE]

Step 4: Build the Grade Submission Template (Reactive Form + Material + Tailwind)

Open `src/app/features/grade-submission/grade-submission.component.html` and bind the `[formGroup]="gradeForm"` with `formControlName` bindings, validation error messages (`<mat-error>`), and button disability states:

```

<div class="max-w-md mx-auto my-8">
  <mat-card class="shadow-xl rounded-2xl bg-slate-900 border
border-slate-800 text-slate-100 p-6">
    <mat-card-header class="mb-4">
      <mat-card-title class="text-xl font-bold text-slate-
100">Grade Submission Form</mat-card-title>
      <mat-card-subtitle class="text-slate-400 text-sm">Instructor
Midterm Grading</mat-card-subtitle>
    </mat-card-header>

    <form [formGroup]="gradeForm" (ngSubmit)="onSubmit()">
      <mat-card-content class="space-y-4">
        <mat-form-field appearance="outline" class="w-full">
          <mat-label>Student ID</mat-label>
          <input matInput type="number" formControlName="studentId"
/>

          @if (gradeForm.controls.studentId.hasError('required')) {
            <mat-error>Student ID is required</mat-error>

```

```

    }
  </mat-form-field>

  <mat-form-field appearance="outline" class="w-full">
    <mat-label>Course ID</mat-label>
    <input matInput type="number" formControlName="courseId"
  />

    @if (gradeForm.controls.courseId.hasError('required')) {
      <mat-error>Course ID is required</mat-error>
    }
  </mat-form-field>

  <mat-form-field appearance="outline" class="w-full">
    <mat-label>Score (0-100)</mat-label>
    <input matInput type="number" formControlName="score" />
    @if (gradeForm.controls.score.hasError('min') ||
gradeForm.controls.score.hasError('max')) {
      <mat-error>Score must be between 0 and 100</mat-error>
    }
  </mat-form-field>

  @if (isSubmitting) {
    <div class="flex justify-center py-3">
      <mat-spinner diameter="32"></mat-spinner>
    </div>
  }

  @if (submissionStatus) {
    <div class="mt-4 p-3 rounded-lg bg-slate-800 text-sky-400
text-sm font-medium border border-slate-700">
      {{ submissionStatus }}
    </div>
  }
</mat-card-content>

<mat-card-actions class="mt-4">
  <button
    mat-raised-button
    color="primary"
    type="submit"
    [disabled]="gradeForm.invalid || isSubmitting"
    class="w-full py-3 text-base font-semibold">
    Submit Final Grade
  </button>
</mat-card-actions>
</form>
</mat-card>
</div>

```

[!NOTE]

Step 5: Add Route Registration

Open `src/app/app.routes.ts` and add the lazy-loaded route configuration parameter inside the routing array:

```
{
  path: 'grade-submission',
  loadComponent: () =>
    import('./features/grade-submission/grade-
    submission.component')
    .then(m => m.GradeSubmissionComponent)
}
```

Exercise 4 Verification and Testing Checklist

Follow these verification steps in order to confirm your request-throttling defensive stream architecture:

1. **Initialize the Frontend Workspace:** Start the local development server (`ng serve`) and navigate your browser window to `http://localhost:4200/grade-submission`.
2. **Validate Form Constraints:** Try submitting invalid values (e.g., score set to `150` or an empty student ID)—observe reactive `<mat-error>` messages and the disabled Submit button.
3. **Simulate a Slow Network Connection:** Open Chrome DevTools, head to the **Network** tab, and set the network throttling dropdown profile selector directly to **Slow 3G** (simulating slow server response times).
4. **Trigger a Local Traffic Burst:** Click the **Submit Final Grade** button rapidly 10 times in a row.
5. **Inspect Outbound Network Telemetry:** Review the logged trace streams inside your browser tab window. You will observe exactly **one single POST request** to `/api/grades`, while the Material spinner provides visual loading feedback. The subsequent 9 clicks are completely ignored by your `exhaustMap` pipeline because they occurred while the primary operation was still actively in flight.